

PulseHR Repository Report

Updated implementation status and verification report

Human Resource Management System for Indian SMEs

Prepared from the HRMS2-main source tree, planning documents, automated tests, Docker smoke checks and browser verification.

Executive summary

PulseHR began as two substantial but disconnected applications: a React/Vite frontend backed entirely by mock data and a Node/Express/Prisma backend with real business logic. The current implementation has joined the first vertical slice end to end: users can authenticate against the real API, roles come from the backend, and the leave module reads and writes the database.

The payroll placeholder has also been removed. Payroll now reads an employee-specific stored salary structure, while PF, TDS, HRA and net-pay calculations remain explicit and tested. The remaining frontend modules have endpoint contracts and reusable query helpers, but several screens still need their JSX-specific adapters and mutation forms.

Current release position: authentication and leave are production-shaped vertical slices; payroll is database-backed; the rest of the module rollout is scaffolded and partially connected.

Repository at a glance

Area	Current state	Evidence
Frontend	React 18, TypeScript, Vite, TanStack Query. Login, protected routing, leave, directory, profile and dashboard identity use real API data.	Frontend build passes; 15 tests pass.
Backend	Express, Prisma, PostgreSQL, Redis. Auth, leave and payroll flows execute against the database.	Backend build passes; 7 tests pass.
Database	Prisma schema includes a new SalaryStructure model linked one-to-one to Employee.	Prisma generation and Docker startup pass.
Runtime	Docker Compose runs frontend, backend, PostgreSQL and Redis.	Containers healthy; browser smoke checks pass.

Implemented changes

Authentication

The frontend now has a real login form at /login. Access tokens stay in module memory, refresh tokens use the backend HttpOnly cookie, and the API client attaches bearer authorization. A 401 triggers one shared refresh operation for concurrent requests and retries the original request once. A failed refresh or second 401 clears the session and React Query cache. The previous demo role switcher is gone; navigation reflects the authenticated role returned by the backend.

Leave vertical slice

Leave balances and requests use TanStack Query. Users can apply for leave, filter and paginate requests, cancel pending requests, view team leave where authorized, and see loading and retryable error states. The dashboard uses live leave balances and request totals. A Docker smoke test verifies database create, read, cancellation and balance restoration.

Payroll salary source

SalaryStructure stores monthly gross pay and calculation parameters per employee. Seed data creates a default structure for each demo account. Payroll processing fails clearly when a structure is missing instead of inventing a salary from an internal ID. The pure calculator supports custom basic, HRA, PF, TDS and professional-tax values.

Reusable module layer

The frontend includes an endpoint map and generic query/mutation helpers for attendance, payslips, documents, expenses, helpdesk, feed, directory, employees, recruitment, onboarding, payroll, analytics and admin settings. Directory and profile are already connected to the real API.

Verification results

Check	Result	Scope
Frontend production build	PASS	Vite bundle generated successfully.
Frontend type-check	PASS	tsc --noEmit -p tsconfig.app.json.
Frontend tests	PASS - 15 tests	API transport, refresh concurrency, timeout, role restoration, logout cache clearing.
Backend production build	PASS	TypeScript compilation and Prisma client generation.
Backend tests	PASS - 7 tests	Refresh cookie transport and salary calculations.
Auth Docker smoke test	PASS - all four roles	Login, identity, role enforcement, refresh rotation, replay rejection and logout revocation.
Leave Docker smoke test	PASS - employee flow	Create, read, cancel and balance restoration.
Payroll Docker smoke test	PASS - 8 seeded employees	Process run with stored salary structures and aggregate totals.
Browser verification	PASS - session lifecycle	Protected redirect, real sign-in, reload restoration, sign-out and signed-out reload.

Security and data handling

The access token is not persisted in `localStorage` or `sessionStorage`. Refresh rotation is enforced by the backend session table, and logout blocklists the access token in Redis. Existing Helmet, rate limiting, audit middleware, Zod validation and role middleware remain active. Sensitive field encryption and masking have not yet been added and should be treated as a release blocker before production use.

Remaining roadmap

Workstream	Status and next action
Phase 3 UI rollout	Wire the existing screens for attendance, payslips, documents, expenses, helpdesk, feed, org chart, recruitment, onboarding, analytics and admin settings to the endpoint map. Remove each module's mock-data imports after its read and mutation flows are verified.
Phase 5 coverage	Auth and payroll now have baseline coverage. Add integration cases for protected resource ownership and payroll edge cases such as overtime, LOP and month boundaries.

Workstream	Status and next action
Phase 6 security	Implement TOTP MFA, Google/Microsoft SSO, IP allow-list enforcement, and field-level encryption/masking for PAN and bank account numbers.
Production hardening	Run migrations instead of db push, configure real secrets, verify HTTPS cookie behavior, set up observability, and perform a security review.

Demo accounts

The development seed uses **Password@123** for **admin@pulsehr.dev**, **hr@pulsehr.dev**, **manager@pulsehr.dev** and **emp1@pulsehr.dev**. These credentials are for the disposable local environment only and must not be reused in any shared or production system.